

The Flappy Bird Project

Overview

We will explain strengths and limitations at the end

- we had divided the code into the following modules:
 - main: contains a high-level runtime life cycle
 - settings: mostly stores game settings
 - runtime_globals: stores shared variables
 - io_init_and_kb: initializes I/O and has keyboard press processing functionalities
 - draw: for drawing frames, it draws birds and pipes, the latter has rasterization hooks for generation and cleanup of pipes
 - update: processes logical events, with update() being called each frame
 - utils: made to contain loosely coupled functions, currently has one function

The flappy bird coordinate system

----->

|

|

V

A top-down coordinate system is used, the position of objects stored as the upper-leftmost character

Key Structures

- The pipe dictionary:
 - x: the pipe's x offset
 - y: the pipe's y offset (the y line matching the higher ----- line of the pipe)
 - passed: whether the bird had already moved passed the pipe
- The canvas: a 2d list where each element is a list of characters representing a row, we used a list of characters as opposed to a string for fast and simple access of pixels (simple because strings are immutable, while lists aren't)

Main.py (by order of execution)

- initialize():
 - Clears the screen (to not have leftover characters from the previous commands)
 - Generates the first pipe(the rest are generated when a pipe moves fully into the canvas (detection is in draw.py))
 - Reads the high score
- Note: it does logical initialisation, while io_init_and_kb does os-specific initialization i/o initialisation
- load_high_score():
 - Returns the high score form highscore.txt, if the file does not exist, it returns 0 and writes '0' to the file

Main.py (by order of execution)

- `main_loop()`:
 - Draws the game scene
 - Updates the game state (by calling `update()`)
 - Processes the last key press since the last frame's end
 - Sleeps for for $1/\text{framerate}$ many seconds (which roughly means that the loop will run `framerate` many times per second)
- the loop stops if `update()` returns `False`, as that indicates the end of the game in our case
- `show_game_over_screen()`: basically waits for the user to press a key before quitting

settings.py

- debug: this is a flag covering whether debug mode is on or not, if True gravity would be disabled and the tester can step-through the game based on key presses, it can be used for collision testing and score update testing, if False, the normal game runs
- width: the width of the game in characters (int)
- height: the height of the game in characters, it was set in a way that maintains a 16:9 ratio based on width (there is a division over 2 because in most terminals the height of a line is the same as the width of two characters) (int)

settings.py

- `frame_rate`: the maximum amount of times the main loop would run per second
- `scroll_speed`: the speed of the pipes (the pipes move backward while the bird is stationary) (int)
- `bird_x`: bird's x position (int)
- `bird_acceleration`: gravity (float)
- `bird_jump_velocity`: the velocity the bird gets if it jumps (float)
- `bird_image`: the characters representing the bird, it mimics the canvas structure, note that it's not as flexible as the code implies, collision has the bird's dimensions hard-coded

settings.py

- `min_pipe_distance_slack_factor`: effectively sets the ratio of the minimum estimated viable pipe distance, and the actual minimum pipe distance used (float)
 - `max_to_min_pipe_distance_factor`: the ratio of the minimum pipe distance and the maximum pipe distance, this is not absolute; it is clamped due to limitations in pipe events (float)
 - `pipe_width`: the width of the pipe (int)
 - `gap_height`: the height of the gap + 2 (as it includes the ----- lines) (int)
-
- A hook in programming often means a function to be called at an event
 - `shift_bird_y(sh)`: used for hooks in `debug_key_actions`
 - `debug_key_actions`: a dictionary of the hooks set in debug mode where the key of the dictionary is the key press as a character, w moves the bird up, s moves the bird down, d moves the pipes backwards (as if the bird moves forwards)

settings.py

- `set_bird_vel(v)`: used for `normal_key_actions`
- `debug_key_actions`: a dictionary of the hooks set in normal mode(`debug = False` mode) where the key of the dictionary is the key press as a character, space sets the bird's velocity to the `bird_jump_velocity` setting, resetting the acceleration build-up in the process
- `key_actions`: holds a pointer to the active key hooks dictionary (selected based on the debug flag)

runtime_globals.py

- score: current score
- high_score: current high score updated as soon as the high score is broken
- pipes: a list of active pipe instances (each pipe is as described in the key structures section)
- bird_y: the current line the bird is at
- bird_velocity: the current velocity of the bird

io_init_and_kb.py

- Note: we are not OS API experts, nor python experts, and as such, we had used help to write this module
- As soon as the module loads it tries to import the msvcrt module, a Windows™-only module, if we are in unix, the import will throw an exception, skipping the windows only code, otherwise *nix specific code will run
 - Note: *nix basically means unix-likes, which includes linux, mac os and the bsd's though we had only tested code on linux
- 1. Windows™ stub: we 'or' the current console mode for stdout state with 100b, which enables virtual terminal processing, this allows us to use terminal-controlling ascii escape sequences

io_init_and_kb.py

2. *nix stub:

- We set a `is_unix` flag to `True`, this is useful for other functions
- It sets the standard input to be non-blocking this makes reads on `stdin` return instantly (even if there is no input)
- We store terminal state before setting standard input to have `cbreak` mode, `cbreak` mode allows reads to return characters as soon as a character is inserted, not as soon as enter is pressed
- We also register a hook on `exit` so that we can reset the terminal to the state just stored

io_init_and_kb.py

- `get_last_ch()`:
 - This function returns the last character from the relevant queues (which basically store keypresses), this function clears the queues in the process, effectively this function returns the last character pressed since the last time this function was called (if any were pressed..)
 - If no 'supported characters' are pressed (or if none at all were pressed) the function will return an empty string
- If we are in *nix os-es we use `read()` and return the last character from the returned string
- If we are on windows we run a loop that drains `getwch`'s queue (by running a while if `kbhit` returns `True`)

io_init_and_kb.py

- `process_ch(ch)`: Checks if the `ch` is mapped in the `settings.key_actions` dictionary, if so it calls the appropriate routine
- `process_last_keypress()`: Calls `process_ch` with the value of `get_last_ch`

draw.py

- `canvas_fill`: the character used to fill the canvas initially
- `canvas`: the only instance of the canvas structure explained earlier (basically a list of elements where each element is a list of characters)
- `clear_terminal()`: it clears the terminal, which makes that the characters present before running the game wiped
- `draw_frame()`: the highest-level `draw_*` function, it:
 - clears the canvas
 - draws the pipes (and runs the rasterization hooks)
 - draws the bird based on the `bird_x` setting and the `bird_y` global
 - prints the canvas

draw.py

- `clear_canvas()`: it clears the canvas, without changing the canvas pointer
- `draw_px(ch, x, y)`: the lowest level `draw_*` function:
 - only overwrites the character at position (x, y) with the character ch if the position is in the boundaries of canvas
 - returns whether the character was in the canvas' bounds or not

draw.py

- `draw_pipes()`:
 - It tries to draw the oldest pipe, if the pipe didn't overwrite any pixel `update.delete_first_pipe()` will be called, deleting the pipe
 - It draws the rest of the pipes but the last
 - It tries to draw the newest pipe, if the pipe did overwrite all the pixels that could be overwritten, then the pipe is fully inside the canvas, and `update.generate_pipe()` would be called, making an even newer pipe

draw.py

- `draw_pipe(canvas, pipe, canvas_width, canvas_height)`: It tries to draw a pipe pair based on the pipe struct passed
 1.
 - draws the top pipe's vertical edges (ones made of the | character) row by row top to bottom
 - draws the top horizontal edge (made of the - character), then the bottom one, both column by column, left to right
 - It draws the bottom pipe's vertical edges (ones made of the | character) row by row top to bottom
 - mind that `gap_height` is inclusive of the horizontal edges
 2. The function compares the number of iterations in loops described in 1. to the number of the pixels successfully overwritten, returns a tuple of whether the numbers match (thus the pipe is fully inside the canvas), and whether there were no overwritten pixels (thus the pipe is fully outside the canvas)

draw.py

- `test_draw_pipe(x, y)`: Shows how would a pipe be rendered at (x, y), and returns the tuple returned by the appropriate `draw_pipe` call
- `draw_bird(x, y)`: `round()`'s y to the nearest integer then calls `draw_px` for each column of each row of `settings.bird_image` with x and y set to the arguments + the offset within the image (effectively it copies `settings.bird_image` at the position (x, y))

draw.py

- `show()`:
 - It sets the terminal cursor to 0, 0 (preventing scrolling, which looked glitchy in Windows™)
 - Prints the score and the high score
 - It joins the elements of the canvas making each element a string representing a line, and joins such strings with new lines
 - It sets a border variable to a string of `settings.width` many asterisks (*)
 - Then it joins the following elements in this order: cursor reset sequence + scores, border, canvas, border, by newlines
 - It flushes stdout just in case

update.py

- update(): this function runs after every frame is drawn, It:
 1. It stores whether the bird had collided in a variable
 2. If the debug mode is off, it would
 - Move the pipes backwards (which gives the illusion that the bird had moved forward)
 - Update the bird's y position based on velocity
 - Update the bird's velocity based on its acceleration
 3. Checks if the score can be increased and updates it if so
 4. Updates the high score (both the variable and the file..) if the score was updated
 5. Returns whether the game could continue (inverts the variable of point 1 then returns it)

update.py

- `move_pipes()`: it shifts every pipe's x position in `runtime_globals.pipes` by `-settings.scroll_speed`
- `increase_score()`: checks every pipe, and if a pipe didn't increase the score and was fully behind the bird, the pipe will be marked, and the score would be increased
- `update_high_score(current_score, current_high_score)`: if `current score > the current highscore`, write the new highscore to `highscore.txt`, it effectively returns the new highscore

update.py

- `check_collision(bird)`: it returns True if:
 1. the bird is above $y=0$, at or below $y=\text{height}$
 2. If the bird is between the vertical lines of any pipe (inclusive), and if its y value is either above or at the pipe's top horizontal line, or is below or at the pipe's bottom horizontal line
 - Otherwise it returns False

update.py/approx_min_pipe_x_off(delta_y)

- Notes:
 - idea and implementation we got without outside help
 - The function isn't 100% accurate, for example: assumes the every relevant point is well a point, it doesn't care that something is a bird/pipe, and it uses round()
 - The derivation has rough edges, mainly regarding signs, the explanation of the signs used in the code is omitted
 - We will refer to the function later as minimum estimated x
- You can say that the function:

update.py/approx_min_pipe_x_off(delta_y)

Takes Δy and returns $x(y)$ which is defined as $x(t) \circ t(y)$

where $x(t)$ is derived like this:

$$scroll_x = \frac{x}{t} \Rightarrow x = t \times scroll_x$$

note: scroll_x is settings.scroll_speed

and $t(y) = y^{-1}(t)$ where $y(t)$ is either:

$$1. velocity_{jump} = \frac{y}{t} \Rightarrow y = velocity_{jump} \times t$$

which is used when Δy is negative, and the bird must jump to reach

$$2. y(t) = \sum_{i=0}^t velocity(i) \text{ where } velocity(i) = acceleration \times i$$

$$y = \sum_{i=0}^t acceleration \times i = acceleration \sum_{i=0}^t i$$

using the triangle number formula:

$$y(t) = \frac{acceleration \times t \times (t + 1)}{2}$$

if we simplify and assume that $t = t + 1$ we get:

$$y(t) = \frac{acceleration \times t^2}{2}$$

an alternate derivation of 2. :

$$y(t) = \int_0^t velocity(i) \times di$$

$$y(t) = \int_0^t acceleration \times i \times di$$

$$y(t) = \frac{acceleration \times t^2}{2} - \frac{acceleration \times 0}{2}$$

$$y(t) = \frac{acceleration \times t^2}{2}$$

update.py/approx_min_pipe_x_off(delta_y)

2. is used when Δy is positive, and the bird needs gravity to go down

now deriving inverses of the equations 1. and 2. :

$$1. y(t) = velocity_{jump} \times t \Rightarrow t = \frac{\Delta y}{velocity_{jump}}$$

$$2. y(t) = \frac{acceleration \times t^2}{2} \Rightarrow \frac{2 \times \Delta y}{acceleration} = t^2$$

$$t = \sqrt{\frac{2 \times \Delta y}{acceleration}}$$

Again, t is got based on either 1. Or 2. Based on arrival is jump bound or gravity bound(eq 1 is arguably simple because acceleration is reset if you keep pressing space), and then x is got based on t

update.py

- generate_pipe():

1. Generates y randomly between [1, settings.height - settings.gap_height - 1)
2. If there are no pipes, there is no delta_y to pass the the min x estimator, and we simply generate a pipe at the edge
3. We store a minimum x offset defined as (minimum estimated x + 1) * slack factor

3.1. We will refer to the stored val as #3

4. We set x to settings.width + a random number between [#3, #3*settings.max_to_min_pipe_distance_factor clamped to settings.width - 1]

4.1. We clamp because otherwise the game may crash as every pipe could be outside the canvas

5. We set the passed flag to false, this is the flag used for score checking

update.py

- `delete_first_pipe()`: removes the head pipe with $O(n)$ time, we could have used a deque, but, the gains are so minimal, and time is short, and we wanted to keep some stuff simple

Strengths

- The code:
 - Cross-platform
 - Somewhat modular and configurable
 - Didn't use any module outside python's standard library
- LLMs and internet had barely helped in logic (just in i/o and draw_pipe, i think), and we wrote even this documentation manually
- We mathematically derived the minimum pipe distance that doesn't make the game impossible (roughly speaking)

Limitations and Weaknesses

- Using rasterization hooks had limited pipe generation and made us unable to safely generate pipes outside the canvas, as they may get deleted
- Making stars outside the canvas had made the bird hitting them inconsistent with the bird hitting the pipes, as the bird becomes invisible in the former case
- We had used $O(n)$ many times where we could have used $O(1)$, eg: in score checking and collision checking, and when removing the head pipe